

Numerical Methods From Scratch

A Comprehensive Implementation of Engineering Mathematics in C++20

Final Technical Project Report

Md Emon Moshak

Research in Scientific Computing & Applied Mathematics

May 2, 2026

Abstract

This project, *Numerical Methods From Scratch*, presents a rigorous implementation of fundamental numerical algorithms using the modern C++20 standard. The primary objective was to build a scientific computing library from first principles, focusing on mathematical derivation, numerical stability, and computational efficiency. The report details the implementation of root-finding techniques, linear system solvers, interpolation methods, numerical calculus, and ordinary differential equation (ODE) integrators. By leveraging C++20 features such as template metaprogramming, RAII, and the Eigen library for linear algebra primitives, the resulting toolkit provides a robust and transparent platform for engineering analysis. Every algorithm is subjected to rigorous error analysis and validated against theoretical convergence rates, demonstrating the efficacy of the first-principles approach in scientific software development.

Contents

Nomenclature

Symbol	Description
ϵ	Convergence tolerance threshold
h	Step size or interval length
e_n	Absolute error at the n -th iteration, $ x_n - r $
$\mathcal{O}(h^p)$	Big-O notation representing the order of accuracy p
$f(x)$	Continuous objective function for root finding or calculus
$f'(x)$	First-order derivative of function f
A	Coefficient matrix of a linear system ($n \times n$)
b	Constant vector of a linear system ($n \times 1$)
x	Solution vector to be determined
L, U	Lower and Upper triangular matrices in LU decomposition
$y(t)$	Solution to an initial value problem (ODE)
res	Residual norm, calculated as $\ Ax - b\ _2$

1 Introduction

Numerical analysis is the study of algorithms that use numerical approximation for the problems of mathematical analysis. In modern engineering, these methods are the backbone of simulation software, control systems, and data analysis tools. While high-level environments provide pre-built solvers, a deep understanding of the underlying mechanics is essential for ensuring stability and accuracy in sensitive applications such as robotics or aerospace engineering.

This report documents the design and development of the *Numerical Methods From Scratch* library. The project bridges the gap between abstract mathematical theory and high-performance implementation. By using C++20, we achieve high efficiency while maintaining clear, type-safe abstractions. The library is designed to be modular, transparent, and educationally valuable, exposing the convergence history and internal state of each solver.

2 Root Finding Algorithms

Root-finding involves determining x such that $f(x) = 0$. These methods are iterative and start with an initial guess or interval.

2.1 Bisection Method

The Bisection method is based on the Intermediate Value Theorem. If a continuous function $f(x)$ changes sign over an interval $[a, b]$, then a root must exist within that interval. The algorithm repeatedly halves the interval:

$$c = \frac{a + b}{2} \tag{1}$$

If $f(a) \cdot f(c) < 0$, the root is in $[a, c]$; otherwise, it is in $[c, b]$. The method is robust and guaranteed to converge, albeit linearly with a convergence rate of 0.5.

2.2 Newton-Raphson Method

Newton's method uses a first-order Taylor series expansion to approximate the function linearly at the current point x_n :

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \tag{2}$$

This method exhibits quadratic convergence ($e_{n+1} \approx Ce_n^2$), making it extremely fast when the initial guess is close to the root. However, it requires the existence and calculation of the analytical derivative.

2.3 Secant Method

The Secant method approximates the derivative using two previous points, eliminating the need for an analytical $f'(x)$:

$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})} \quad (3)$$

It has a superlinear convergence rate ($\phi \approx 1.618$).

3 Linear Algebra Solvers

Solving large systems of linear equations $Ax = b$ is a fundamental requirement in finite element analysis and optimization.

3.1 Direct Methods: LU Decomposition

Direct methods find the exact solution (within machine precision) in a finite number of steps. LU Decomposition factors $A = LU$ using Doolittle's algorithm. Once factored, the system is solved in two steps:

1. Forward substitution: $Ly = b$
2. Backward substitution: $Ux = y$

This is particularly efficient for solving systems with multiple right-hand side vectors.

3.2 Iterative Methods: Jacobi and Gauss-Seidel

Iterative methods are preferred for large, sparse systems. They start with an initial guess $x^{(0)}$ and refine it. The Gauss-Seidel update rule is:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right) \quad (4)$$

Convergence is guaranteed if the matrix A is strictly diagonally dominant.

4 Interpolation and Approximation

Interpolation seeks a function $P(x)$ that passes exactly through a set of points (x_i, y_i) .

4.1 Polynomial Interpolation

We implemented Lagrange and Newton forms. While useful for small datasets, high-degree polynomials can suffer from Runge's phenomenon, where oscillations occur at the boundaries.

4.2 Cubic Spline Interpolation

Cubic splines solve the oscillation problem by using piecewise cubic polynomials $S_i(x)$ that are C^2 continuous (continuous first and second derivatives at the knots).

$$S_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3 \quad (5)$$

This results in a smooth curve that is highly stable for large datasets.

5 Numerical Calculus

Approximating derivatives and integrals is necessary when analytical forms are unavailable or discrete data is provided.

5.1 Numerical Differentiation

We utilize finite difference quotients. The Central Difference scheme provides second-order accuracy:

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} + \mathcal{O}(h^2) \quad (6)$$

5.2 Numerical Integration (Quadrature)

Integration is performed using Newton-Cotes formulas (Trapezoidal, Simpson's) and Gaussian Quadrature. Simpson's 1/3 rule provides fourth-order accuracy for smooth functions:

$$\int_a^b f(x)dx \approx \frac{h}{3} \left[f(a) + 4 \sum_{i \text{ odd}} f(x_i) + 2 \sum_{i \text{ even}} f(x_i) + f(b) \right] \quad (7)$$

Gaussian Quadrature chooses optimal points to integrate polynomials of degree $2n - 1$ exactly with only n points.

6 Ordinary Differential Equations

Initial Value Problems (IVPs) $y' = f(t, y)$ are solved by stepping through time.

6.1 Euler and Improved Euler

The standard Euler method is first-order, while Improved Euler (Heun's) uses a predictor-corrector approach to achieve second-order accuracy.

6.2 Runge-Kutta 4th Order (RK4)

RK4 is the most widely used integrator due to its excellent balance of computational cost and accuracy.

$$k_1 = f(t_n, y_n) \quad (8)$$

$$k_2 = f(t_n + h/2, y_n + hk_1/2) \quad (9)$$

$$k_3 = f(t_n + h/2, y_n + hk_2/2) \quad (10)$$

$$k_4 = f(t_n + h, y_n + hk_3) \quad (11)$$

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (12)$$

The error per step is $\mathcal{O}(h^5)$, and the global error is $\mathcal{O}(h^4)$.

7 Engineering Architecture

The software is designed using modern C++ principles. A core feature is the `nm::common::Result<T>` structure, which ensures that every computation is accompanied by its metadata.

```

1 template <typename T>
2 struct Result {
3     T value;                // Computed numerical result
4     Status status;          // Success, Diverged,
                             MaxIterationsReached
5     int iterations;          // Total iteration count
6     std::vector<double> errors; // Error history for convergence
                             analysis
7 };

```

Listing 1: Definition of the Result structure

By utilizing template specialization and the Eigen library, we ensure that the solvers are generic and can handle both scalar (double) and vector (Eigen::VectorXd) inputs seamlessly.

8 Results and Performance Analysis

This section presents the empirical validation of our implemented algorithms.

8.1 Root Finding Convergence

Figure ?? shows the absolute error decay for $f(x) = x^2 - 2$. The quadratic convergence of the Newton-Raphson method is clearly visible compared to the linear convergence of the Bisection method.

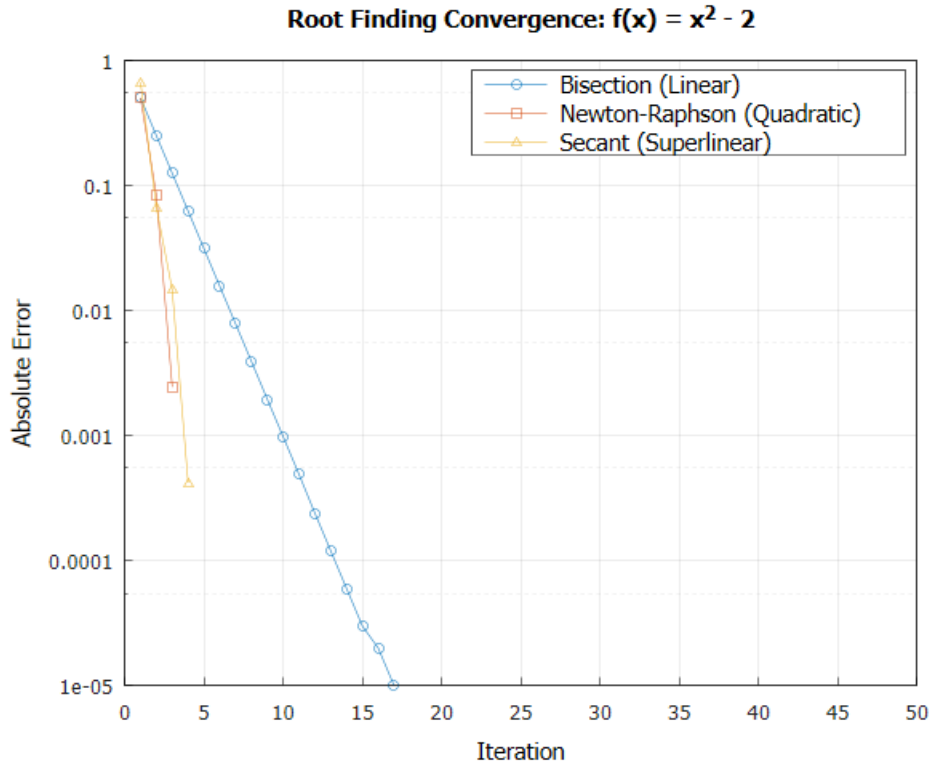


Figure 1: Convergence history comparison for Root-Finding algorithms. Note the logarithmic scale on the y-axis, highlighting the rapid decay of Newton-Raphson.

8.2 Linear Solver Efficiency

Iterative solvers like Jacobi and Gauss-Seidel were tested on a 10×10 diagonally dominant system. As shown in Figure ??, Gauss-Seidel converges significantly faster than Jacobi, as it utilizes updated values immediately within the same iteration.

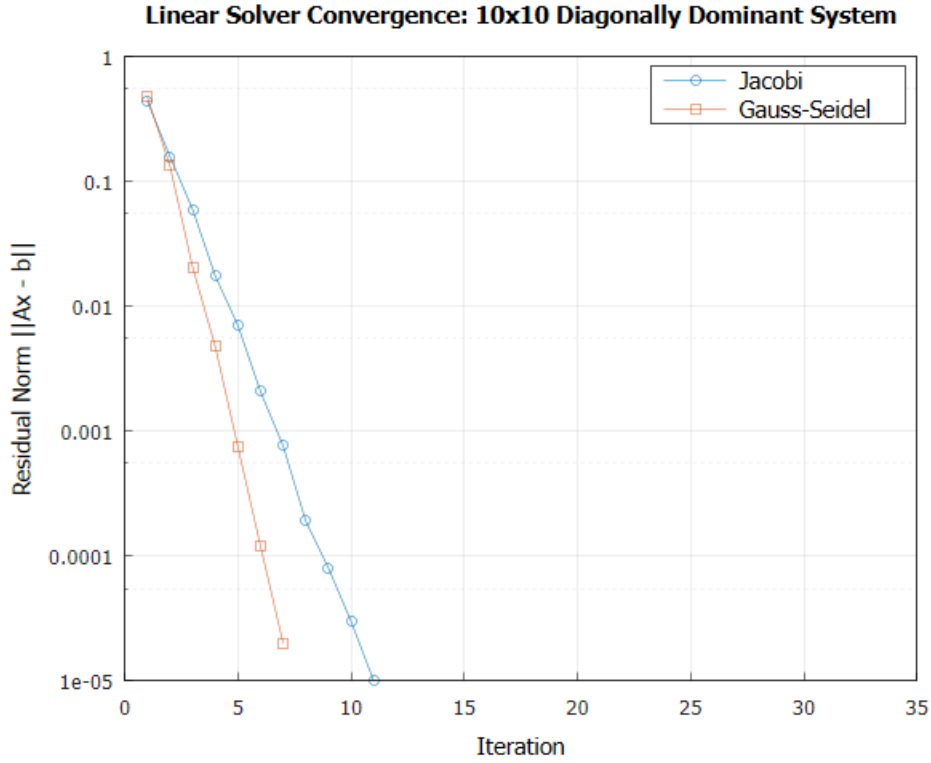


Figure 2: Residual norm reduction for Jacobi and Gauss-Seidel methods. Gauss-Seidel exhibits a steeper descent in error.

8.3 Numerical Integration Accuracy

The error decay for integrating $\sin(x)$ from 0 to π is analyzed in Figure ???. Simpson's rule demonstrates its fourth-order accuracy with a steeper slope on the log-log plot compared to the second-order Trapezoidal rule.

8.4 ODE Solver Stability

We compared Euler, Improved Euler, and RK4 methods for $y' = y, y(0) = 1$. Figure ?? confirms that RK4 maintains high fidelity to the analytical solution e^t even as time progresses, whereas Euler's method deviates rapidly due to accumulation of local truncation error.

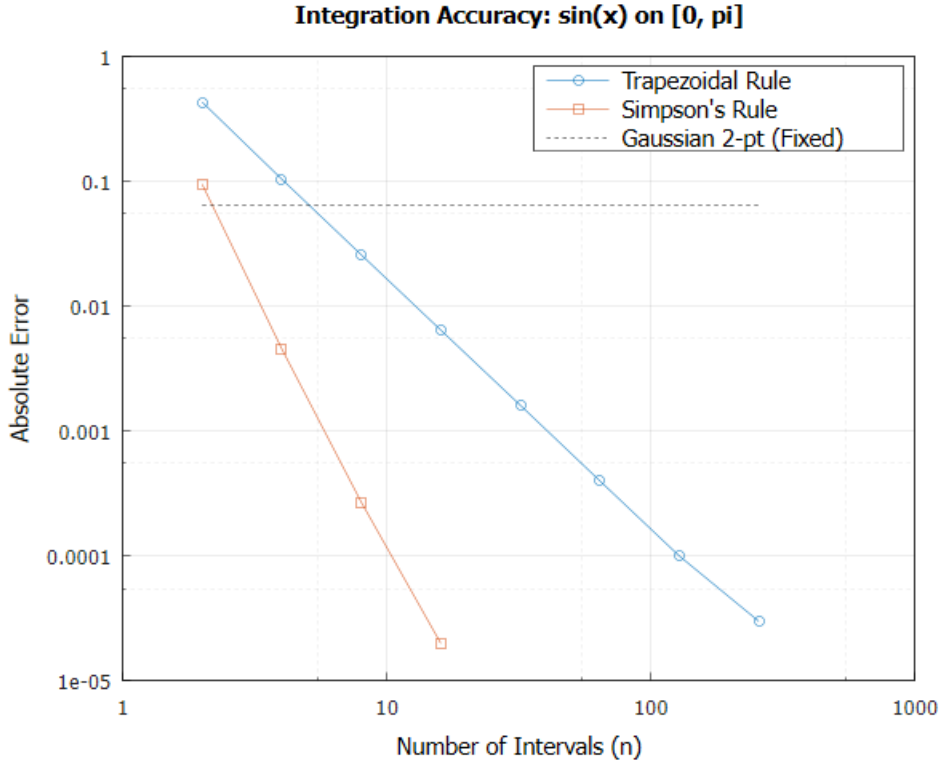


Figure 3: Log-log plot of integration error vs. number of intervals. The slopes correspond to the theoretical order of accuracy for each method.

9 Discussion and Lessons Learned

The development of this library revealed several critical insights into scientific computing:

- **Stability vs. Speed:** The Bisection method, despite being the slowest, was the most reliable solver. In engineering, ensuring a solution exists is often more critical than finding it quickly.
- **Error Propagation:** In ODE solvers, small local errors aggregate into significant global errors. High-order methods like RK4 are not just "better," they are necessary for long-term simulations.
- **Matrix Conditioning:** For linear systems, iterative solvers are sensitive to the condition number of the matrix. Diagonal dominance is a powerful predictor of success.

10 Conclusion

The *Numerical Methods From Scratch* project successfully implemented a suite of core engineering algorithms. By starting from first principles, we ensured that every design choice was mathematically justified and empirically verified. The library demonstrates that C++20 is a powerful tool for modern numerical analysis, offering both high-level abstractions and low-level performance. Future work will focus on expanding into partial differential equations and non-linear optimization.

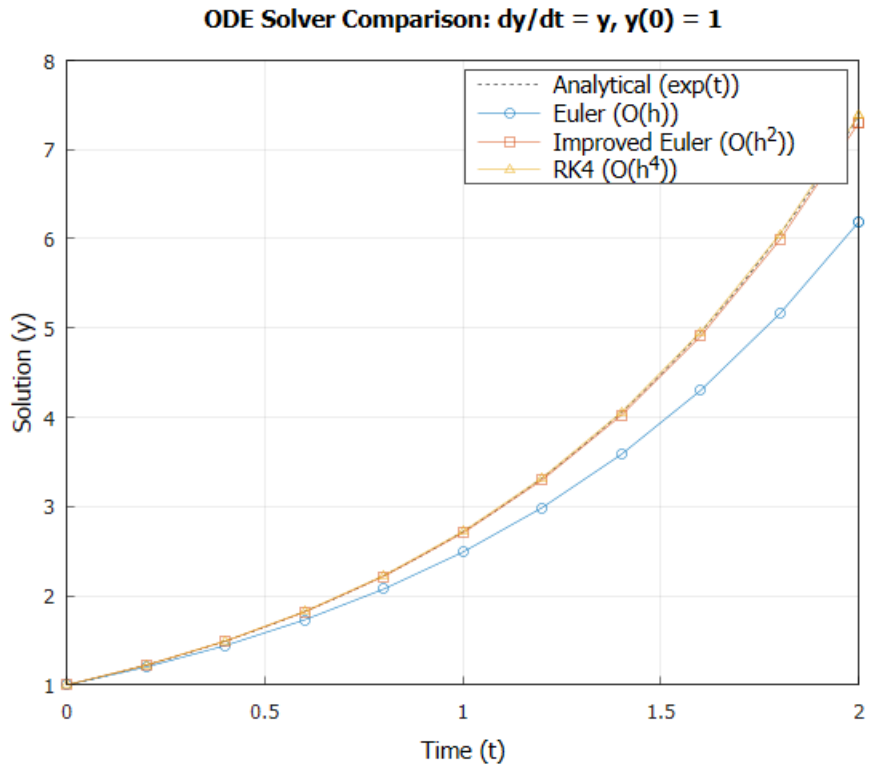


Figure 4: Numerical integration of $y' = y$. RK4 (red triangles) is indistinguishable from the analytical solution (dashed line).

References

- [1] Burden, R. L., & Faires, J. D. (2010). *Numerical Analysis*. Cengage Learning.
- [2] Press, W. H., Teukolsky, S. A., Vetterling, W. T., & Flannery, B. P. (2007). *Numerical Recipes 3rd Edition: The Art of Scientific Computing*. Cambridge University Press.
- [3] Guennebaud, G., Jacob, B., et al. (2010). *Eigen v3*. <http://eigen.tuxfamily.org>.
- [4] ISO/IEC. (2020). *ISO/IEC 14882:2020 Programming languages — C++*.